

Experiment: Change Data Capture (CDC) Pipeline using PostgreSQL, Apache Kafka, Debezium and Docker Compose

Aim

To design and implement a **Change Data Capture (CDC)** pipeline using **PostgreSQL**, **Apache Kafka**, **Debezium Connect**, and **Docker Compose**, and capture database changes (INSERT, UPDATE, DELETE) as real-time Kafka events.

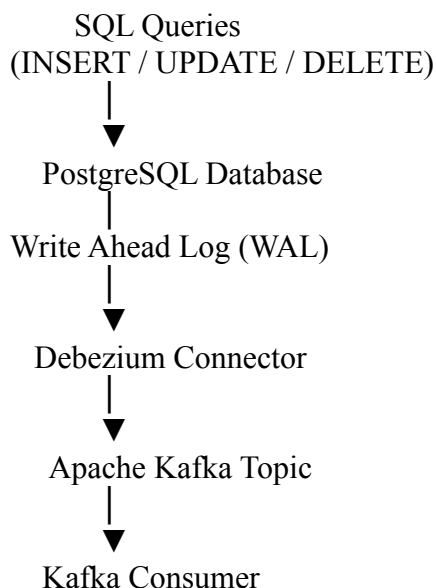
Objective

- Configure PostgreSQL for logical replication.
- Deploy PostgreSQL, Kafka, Zookeeper and Debezium using Docker Compose.
- Create a sample database and table.
- Register a Debezium PostgreSQL connector.
- Capture INSERT, UPDATE and DELETE operations in Kafka.

Software Requirements

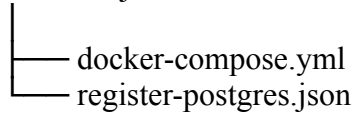
- Docker Desktop / Docker Engine
- Docker Compose
- PostgreSQL 15
- Apache Kafka
- Apache Zookeeper
- Debezium Connect
- Windows Command Prompt / PowerShell / Linux Terminal

Architecture



Project Structure

CDC-Project/



Step 1: Create docker-compose.yml

Create a file named **docker-compose.yml** and add the following configuration.

version: '3.8'

services:

postgres:

image: postgres:15

container_name: postgres

ports:

- "5432:5432"

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

POSTGRES_DB: company

command:

- "postgres"

- "-c"

- "wal_level=logical"

zookeeper:

image: confluentinc/cp-zookeeper:7.5.0

container_name: zookeeper

ports:

- "2181:2181"

environment:

ZOOKEEPER_CLIENT_PORT: 2181

kafka:

image: confluentinc/cp-kafka:7.5.0

container_name: kafka

depends_on:

- zookeeper

ports:

- "9092:9092"

environment:

KAFKA_BROKER_ID: 1

```
KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

connect:

```
image: debezium/connect:2.5
```

```
container_name: debezium
```

depends_on:

- kafka
- postgres

ports:

- "8083:8083"

environment:

```
BOOTSTRAP_SERVERS: kafka:9092
```

```
GROUP_ID: 1
```

```
CONFIG_STORAGE_TOPIC: connect_configs
```

```
OFFSET_STORAGE_TOPIC: connect_offsets
```

```
STATUS_STORAGE_TOPIC: connect_statuses
```

Step 2: Start the Containers

Open the terminal inside the project folder and execute:

```
docker compose up -d
```

Verify all containers:

```
docker ps
```

Expected running containers:

- postgres
- zookeeper
- kafka
- debezium

Step 3: Connect to PostgreSQL

```
docker exec -it postgres psql -U postgres -d company
```

Verify the current database:

```
SELECT current_database();
```

Step 4: Verify Logical Replication

Execute:

```
SHOW wal_level;
```

Expected Output:

logical

Check replication settings:

SHOW max_replication_slots;

SHOW max_wal_senders;

Step 5: Create Employee Table

```
CREATE TABLE employee(  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  salary INT  
);
```

Verify:

\dt

Step 6: Insert Initial Data

```
INSERT INTO employee  
VALUES  
(135848,'Nishantha',25000);
```

Verify:

```
SELECT * FROM employee;
```

Step 7: Create Debezium Connector Configuration

Create a file named **register-postgres.json**

```
{  
  "name": "postgres-connector",  
  "config": {  
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",  
    "database.hostname": "postgres",  
    "database.port": "5432",  
    "database.user": "postgres",  
    "database.password": "postgres",  
    "database.dbname": "company",  
    "database.server.name": "company_server",  
    "topic.prefix": "company_server",  
    "plugin.name": "pgoutput",  
    "table.include.list": "public.employee"  
  }  
}
```

Step 8: Register the Connector

Windows CMD:

```
curl -X POST http://localhost:8083/connectors ^  
-H "Content-Type: application/json" ^  
-d @register-postgres.json
```

Linux/macOS:

```
curl -X POST http://localhost:8083/connectors \  
-H "Content-Type: application/json" \  
-d @register-postgres.json
```

Step 9: Verify Connector Registration

List connectors:

```
curl http://localhost:8083/connectors
```

Expected Output:

```
["postgres-connector"]
```

Check connector status:

```
curl http://localhost:8083/connectors/postgres-connector/status
```

Expected:

Connector State : RUNNING

Task State : RUNNING

Step 10: Start Kafka Consumer

```
docker exec -it kafka kafka-console-consumer \  
--bootstrap-server localhost:9092 \  
--topic company_server.public.employee \  
--from-beginning
```

The consumer waits for database changes.

Step 11: Test INSERT Operation

Execute:

```
INSERT INTO employee  
VALUES(135986,'Kaviya',30000);
```

Kafka displays:

Operation : c (Create)

Step 12: Test UPDATE Operation

```
UPDATE employee  
SET salary=38000  
WHERE id=135848;
```

Kafka displays:

Operation : u (Update)

Step 13: Test DELETE Operation

```
DELETE FROM employee  
WHERE id=135848;
```

Kafka displays:

Operation : d (Delete)

Operation Codes

Co de	Meaning
c	Create (INSERT)
u	Update
d	Delete
r	Snapshot

Expected Output

- Docker containers run successfully.
- PostgreSQL is configured for logical replication.
- Debezium connector is registered and in RUNNING state.
- INSERT, UPDATE and DELETE operations are captured automatically.
- Kafka receives real-time JSON events generated by Debezium.

Result

A Change Data Capture (CDC) pipeline was successfully implemented using PostgreSQL, Apache Kafka, Debezium Connect and Docker Compose. Database changes were captured in real time and streamed to Kafka topics without writing any custom application code.